

Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: Cab Booking System

Team Members and their roles:

Name	Role
Mangalagiri Tejaswi	Team Lead
Putta Harshitha	Member
Deva Charmi Kondapalli	Member
Mohammad Sabir Baig	Member
Orchu Likhitha	Member

2. Project Overview

Purpose

The Cab Booking System is a full-stack web application developed to simplify cab reservation and ride management. The platform allows users to register, log in, book cabs, view booking history, and manage their profiles. Administrators can manage users, drivers, and bookings efficiently. The system provides a user-friendly interface for secure and hassle-free cab booking.

3. Architecture

Frontend

Built using React.js, HTML, CSS, JavaScript and Bootstrap for responsive UI. Axios is used to communicate with the backend APIs

Backend

Developed using Node.js and Express.js. It provides REST APIs for user authentication, cab booking, ride management, and admin operations.

Database

MongoDB is used to store user details, cab information, bookings, and admin data securely.

4. Setup Instructions

Prerequisites

Windows 10/11, Linux or macOS

Node.js (v16+)

npm

React.js

Express.js

MongoDB

Visual Studio Code

Git & GitHub

Postman

Google Chrome

Installation

Clone the repository.

Run npm install in both client and server folders.

Configure MongoDB connection.

Add environment variables.

Start backend server.

Start frontend application.

Recommended Hardware

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better
- **RAM:** minimum 8 GB (16 GB recommended)
- **Storage:** at least 1 GB free space for package installations, MongoDB setup, and local project files
- **Display:** 1366x768 or higher
- **Frontend**
- **npm start**
- **Backend**
- **npm start**

Installation

- Clone the repository from the shared Git/GitHub source.
- Run "npm install" inside the client directory to install frontend dependencies.
- Run "npm install" inside the server directory to install backend dependencies.
- Set up the required environment variables (MongoDB connection string, JWT secret, port number).
- Start MongoDB locally or configure a connection to a hosted MongoDB instance.

5. Folder Structure

Client (React Frontend)

The client directory contains the React application, organized into components, pages, services (for Axios API calls), assets, and routing configuration that together implement navigation bars, booking forms, profile pages, and appointment lists.

Server (Node.js Backend)

The server directory contains the Express.js application, organized into routes, controllers, models (Mongoose schemas for Users, Doctors, and Appointments), middleware (authentication and validation), and configuration files.

6. Running the Application

- **Frontend:** run "npm start" in the client directory.
- **Backend:** run "npm start" in the server directory.

7. API Documentation

The backend exposes RESTful endpoints across the following modules. Each endpoint should be documented with its request method, parameters, and example responses:

- User Registration and Login
- Doctor Browsing and Profile Viewing
- Appointment Scheduling and Management
- Secure Document Upload (Reports/Prescriptions)
- Admin Dashboard for Doctor Approval and Platform Management

Full CRUD operations, routing, and validation are implemented across all entities to ensure smooth user interaction and data consistency.

8. Authentication

- JWT (JSON Web Tokens) is used for authentication and authorization.
- User, doctor, and admin roles are defined with access control on endpoints.
- Passwords are encrypted using bcrypt and sessions are managed securely.
- Admin functionalities such as doctor verification and system monitoring are restricted to authorized personnel only.

9. User Interface

The interface is designed using React.js and Bootstrap for responsive layouts, covering navigation bars, booking forms, profile pages, and appointment lists across the following workspace areas: Project Architecture, Project Setup and Configuration, Backend Development, Database Development, Frontend Development, and Project Execution.

(Insert screenshots or GIFs showcasing the different UI features here.)

10. Testing

- Test all CRUD modules, authentication, and appointment scheduling.
- Validate appointment booking accuracy, data persistence, and file uploads.
- Test API responses using Postman to ensure proper request handling.
- Ensure responsiveness across multiple devices and browsers for accessibility.

11. Screenshots or Demo

Refer to the Demo Screenshots and Drive Links captured under the Project Execution phase for screenshots and a link to a live/recorded demo showcasing the application.

12. Known Issues

No known bugs or issues have been logged at this stage. This section should be updated by the team as issues are identified during development and testing.

13. Future Enhancements

- Integrate an online payment gateway for consultation fees.
- Add video/audio consultation support between patients and doctors.
- Introduce real-time chat and push notifications.
- Build a mobile application version of the platform.
- Add doctor ratings and patient reviews.
- Optimize MongoDB queries and enable structured logging/error handling within Express.js for better debugging and maintenance.

Project Stats

Total Epics: 6

Total Tasks: 19

Total Subtasks: 0

Skills Required: CSS, Node.js, Express.js, React.js, MongoDB, JavaScript, Html